NutriGuide: Complete Technical Report & Interview Master Guide

Table of Contents

SECTION 1 — COMPLETE PROJECT OVERVIEW

Project Name: NutriGuide **Project Purpose:** To provide a comprehensive web platform where users can discover personalized, healthy recipes tailored to their profile and track their daily meals, calories, and macros.

Real-world problem solved: Many individuals struggle to find meals that align with specific health goals (weight loss, muscle gain) and dietary restrictions (vegan, vegetarian). Tracking macros and calories is often tedious. NutriGuide automates recipe discovery and meal tracking in a centralized platform.

Key Features: - **User Profiling:** Captures Age, Weight, Target Weight, Goal, Diet Preference, Allergies. - **Personalized Recommendations:** Rule-based recommendation engine prioritizing recipes based on dietary needs and caloric limits. - **Meal Tracking:** Daily caloric counting with visual progress dashboards (Recharts). - **Admin Panel:** Complete CRUD management of the recipe database. - **Analytics:** Data-rich historical insights into diet trends.

Application Workflow: 1. **Frontend Workflow:** Users interact with a React SPA (Single Page Application). React Router handles navigation seamlessly

without reloading. Context API manages the auth state globally. 2. **Backend Workflow:** Express server receives RESTful calls. Controllers extract data, communicate with MySQL via raw SQL pooling, and return JSON. 3. **Authentication Workflow:** JWT (JSON Web Tokens) generated upon login. The token is sent in the `Authorization: Bearer` header for protected API routes.

Industry Relevance: HealthTech is a rapidly growing sector. Applications dealing with user-specific health tracking, data visualization, and secure authentication are highly relevant.

SECTION 2 — COMPLETE TECHNOLOGY EXTRACTION

1. React (v19)

- **Role:** Frontend UI Library.
- **Why chosen:** Component-based architecture allows for reusable UI pieces (e.g., RecipeCard, Navbar). Virtual DOM provides high performance.
- **Usage:** Entire `frontend/src` directory.
- **Interview Importance:** Very High. Understand Hooks (useState, useEffect, useContext), Virtual DOM, and state lifting.

2. Node.js & Express.js (v5)

- **Role:** Backend runtime and web framework.
- **Why chosen:** JavaScript everywhere. Express is lightweight and perfect for REST APIs.
- **Usage:** Entire `backend` directory.
- **Interview Importance:** High. Understand middleware, event loop, asynchronous non-blocking I/O.

3. MySQL (mysql2)

- **Role:** Relational Database.
- **Why chosen:** Structured data (users, recipes, logs) with clear relationships. ACID compliance ensures reliable meal tracking.
- **Usage:** Backend database.
- **Common Viva Question:** Why MySQL and not MongoDB? (Answer: The data is highly relational. A user has many meal logs, each meal log belongs to one recipe. Relational databases enforce these constraints via Foreign Keys better than NoSQL.)

4. **JSON Web Tokens (jsonwebtoken)**

- **Role:** Stateless Authentication.
- **How it works:** After verifying credentials, backend signs a token containing the User ID and Role. Frontend stores it in `localStorage` and sends it with every request.
- **Security:** Signed via `JWT_SECRET`. Prevents tampering.

5. **Bcrypt.js**

- **Role:** Password Hashing.
- **How it works:** Hashes user passwords with a salt before storing in MySQL. Prevents plaintext password leaks during database breaches.

6. **Vite**

- **Role:** Frontend Build Tool.
- **Why chosen:** Significantly faster Hot Module Replacement (HMR) and build times compared to Create React App (Webpack).

7. **Recharts**

- **Role:** Data Visualization.
- **Usage:** Used in the Analytics page to plot caloric intake over time.

SECTION 3 — FRONTEND COMPLETE ANALYSIS

Architecture: Component-Based Architecture (CBA). **Routing:** `react-router-dom` handles client-side routing. Includes a `<ProtectedRoute>` wrapper that verifies the `AuthContext`. **State Management:** React Context API (`AuthContext.jsx`) is used to store the user object globally, avoiding prop drilling. **Styling:** Custom CSS with Glassmorphism UI elements (translucent backgrounds, blurs) for a modern, premium aesthetic.

Core Pages: 1. **Landing / Login / Register:** Unauthenticated views. 2. **Dashboard:** Primary hub for users showing today's progress and recommendations. 3. **Meals:** Search and filter the entire recipe database. 4. **Progress & Analytics:** Data visualization of logged meals. 5. **Admin Panel:** Table-based view for administrators to add/remove recipes.

Performance Optimization: - Uses modern functional components. - React Router prevents full page reloads.

Frontend Security: - JWT is checked before rendering protected routes. - Forms are controlled components to sanitize basic inputs.

SECTION 4 — BACKEND COMPLETE ANALYSIS

Architecture: MVC (Model-View-Controller) adapted for APIs (Controller-Route-Service). - `routes/`: Define API endpoints and attach middleware. - `controllers/`: Handle business logic (req, res). - `middlewares/`: `auth.js` validates JWT tokens.

API Architecture: RESTful. - Stateless communication. - Uses standard HTTP methods (GET, POST, PUT, DELETE).

Authentication Flow: `POST /api/auth/login` -> Validate Email -> `bcrypt.compare` Password -> Generate JWT -> Return Token.

Recommendation Logic (`recipeController.js`): A rule-based algorithm: 1. Fetches user profile (diet preference, health goal, allergies). 2. Queries recipes matching `diet_tag`. 3. Filters out recipes containing allergic ingredients via string matching/JSON parsing. 4. Sorts by calories (Ascending for weight loss, Descending for muscle gain). 5. Returns Top 10.

SECTION 5 — DATABASE COMPLETE ANALYSIS

Database Type: MySQL Relational Database.

ER Schema & Tables

1. **users:**
   - `id` (PK), `email` (Unique), `password` (Hashed), `role` (user/admin).
   - Profile data: `age`, `weight`, `diet_preference`, `health_goal`, `allergies`.
2. **recipes:**
   - `id` (PK), `title`, `description`, `ingredients` (JSON), `calories`, macros, `diet_tag`.
3. **meal_logs:**
   - `id` (PK), `user_id` (FK -> users), `recipe_id` (FK -> recipes), `date`, `meal_type`.

Relationships: - 1 User -> N Meal Logs (One-to-Many). - 1 Recipe -> N Meal Logs (One-to-Many). - `ON DELETE CASCADE`: If a user is deleted, their meal logs are automatically wiped.

Query Optimization: Use of foreign keys ensures referential integrity.

SECTION 6 — API COMPLETE ANALYSIS

1. Authentication APIs: - `POST /api/auth/register`: Creates new user. Hashes password. - `POST /api/auth/login`: Authenticates user. Returns JWT.

- `PUT /api/auth/profile`: Updates user weight, goals, allergies.

2. Recipe APIs: - `GET /api/recipes`: Fetches all recipes. - `GET /api/recipes/recommendations`: Triggers the recommendation algorithm. Requires JWT. - `POST /api/recipes`: (Admin Only) Creates a new recipe. - `DELETE /api/recipes/:id`: (Admin Only) Deletes a recipe.

3. Meal APIs: - `POST /api/meals`: Logs a recipe eaten by the user for a specific date. - `GET /api/meals/today`: Fetches user's meals for the current date. - `GET /api/meals/history`: Fetches historical data for Recharts analytics.

SECTION 7 — AUTHENTICATION & SECURITY ANALYSIS

Security Implementations: 1. **JWT (Stateless Auth):** No server-side sessions. The server validates the token mathematically using `JWT_SECRET`. 2. **Password Hashing:** `bcryptjs` with salt rounds = 10. 3. **Role-Based Access Control (RBAC):** Backend checks `req.user.role === 'admin'` before allowing recipe creation/deletion. Frontend hides the "Admin Panel" link for standard users. 4. **CORS:** Configured in `server.js` to prevent unauthorized domains from calling the API. 5. **SQL Injection Prevention:** Uses `mysql2` parameterized queries (`?`). E.g., `SELECT * FROM users WHERE email = ?`. This prevents attackers from injecting malicious SQL logic.

SECTION 8 — DEPLOYMENT & DEVOPS ANALYSIS

Since this is local development, below is the standard deployment strategy for this stack. - **Frontend (Vite/React):** Build using `npm run build`. The `dist` folder can be hosted on Vercel, Netlify, or AWS S3. - **Backend (Node/Express):** Hosted on platforms like Render, Heroku, or AWS EC2. Requires Node runtime. - **Database (MySQL):** Managed services like AWS RDS, PlanetScale, or Aiven.

Environment Variables: `.env` hides secrets like `DB_PASSWORD` and `JWT_SECRET`. These are injected at runtime in production.

SECTION 9 — COMPLETE FILE & FOLDER STRUCTURE ANALYSIS

```
NutriGuide/
  backend/
      config/db.js           # MySQL connection pool setup
      controllers/           # Core business logic (auth, recipes, meals)
```

```
    middlewares/auth.js    # JWT verification logic
    routes/                # Express router linking endpoints to controllers
    server.js              # Application entry point, CORS, express.json()
    .env                   # Environment secrets
    package.json           # Backend dependencies
frontend/
    src/
        components/        # Reusable UI (Navbar)
        context/           # AuthContext for global state
        pages/             # Route views (Login, Dashboard, Analytics)
        App.jsx            # React Router setup, Protected Routes
        index.css          # Global styling, variables, glassmorphism
    package.json           # Frontend dependencies (React, Vite, Recharts)
```

SECTION 10 — COMPLETE CODE FLOW EXPLANATION

Scenario: User Logs a Meal 1. **UI Interaction:** User clicks "Add to Tracker" on a recipe card in `Dashboard.jsx`. 2. **Frontend Call:** `axios.post('/api/meals', { recipeId, mealType, date })` is fired. The JWT token is attached in headers. 3. **Backend Middleware:** Express routes it through `middlewares/auth.js`. Token is decoded. `req.userId` is attached. 4. **Controller Logic:** `mealController.js` validates the data and executes a parameterized `INSERT INTO meal_logs` SQL query. 5. **Database:** MySQL saves the record. 6. **Response:** Controller sends `201 Created`. 7. **UI Update:** Frontend fetches updated `GET /api/meals/today`, state updates, and the UI progress bar increments automatically.

SECTION 11 — COMPLETE INTERVIEW PREPARATION

Q1: What is the Virtual DOM in React? *Answer:* The Virtual DOM is an in-memory representation of the real DOM. React compares the Virtual DOM with a snapshot of the previous Virtual DOM (diffing) and updates only the changed elements in the real DOM, making UI updates highly efficient.

Q2: How did you implement authentication? *Answer:* I used JWT (JSON Web Tokens). Upon successful login, the backend generates a token signed with a secret key and sends it to the client. The client stores it in local storage and attaches it to the Authorization header for protected API requests. The backend verifies this token using middleware.

Q3: Explain your database schema. *Answer:* I used a relational MySQL database with three main tables: users, recipes, and meal_logs. `meal_logs` acts

as a junction-like table containing foreign keys linking a `user_id` to a `recipe_id`, allowing a one-to-many relationship tracking user consumption.

Q4: How does bcrypt work? *Answer:* Bcrypt is a one-way hashing algorithm. It adds a random string (salt) to the password before hashing it. This prevents attackers from using Rainbow Tables to crack passwords if the database is compromised.

Q5: What are parameterized queries? *Answer:* Parameterized queries (using `?` in `mysql2`) separate SQL logic from user input. The database driver escapes the input automatically, completely preventing SQL Injection attacks.

SECTION 12 — CDAC VIVA SPECIAL PREPARATION

Faculty: Why did you choose React over plain HTML/JS? **You:** React's component-based architecture allowed me to build reusable UI elements like Recipe Cards and Navbars. Furthermore, its client-side routing makes the app a Single Page Application (SPA), meaning page transitions are instant without server reloads, providing a much better user experience.

Faculty: How does the Recommendation Algorithm work? Did you use Machine Learning? **You:** No, for this scope, I used a deterministic rule-based algorithm. The backend queries the user's profile, fetches recipes matching their diet tag (e.g., vegan), filters out any recipes containing ingredients listed in the user's allergies, and finally sorts them by calories based on whether the user wants to lose weight or gain muscle.

Faculty: Explain the Request-Response lifecycle when I open the Dashboard. **You:** 1. The React `Dashboard` component mounts and triggers `useEffect`. 2. An Axios `GET` request is fired to `/api/recipes/recommendations` with the JWT in the header. 3. Node/Express receives the request. The `auth` middleware intercepts, verifies the JWT signature, extracts `userId`, and passes control to the controller. 4. The controller runs SQL queries against MySQL to fetch personalized recipes. 5. JSON is returned to the frontend. 6. React updates state, and the Virtual DOM re-renders the UI to display the recipe cards.

SECTION 13 — HR INTERVIEW PREPARATION

Q: Explain your project to a non-technical person. *Answer:* NutriGuide is a health app. You tell it your goals—like losing weight or building muscle—and any allergies you have. It instantly recommends meals that fit your diet. You can also log what you eat, and the app shows you beautiful charts of your daily progress.

Q: What challenges did you face? *Answer:* Handling the asynchronous

nature of JavaScript during database calls was challenging initially. Also, designing a recommendation algorithm that dynamically filters out text-based allergens from a JSON ingredient list required careful string manipulation and error handling.

SECTION 14 — DEBUGGING & TROUBLESHOOTING

- **Bug:** `CORS error` on frontend API calls.
  - **Fix:** Used the `cors` package in Express `app.use(cors())` to allow cross-origin requests.
- **Bug:** JWT signature invalid.
  - **Fix:** Ensured the `.env JWT_SECRET` matched exactly on token generation and token verification.
- **Bug:** React State not updating on screen.
  - **Fix:** React state is immutable. Used spread operator `...` to create new arrays/objects instead of mutating existing ones directly.

SECTION 15 — PERFORMANCE & OPTIMIZATION

- **Frontend:** Used React Router for SPA (no page reloads). Used Vite for fast bundling.
- **Backend:** Database connection pooling (`mysql2/promise`) prevents opening a new DB connection for every request, drastically improving API response times under load.
- **Database:** Added Indexes on heavily queried columns (e.g., `email` in users, `diet_tag` in recipes).

SECTION 16 — TESTING ANALYSIS

- **API Testing:** Used Postman to test all REST endpoints independently of the frontend, ensuring proper status codes (200, 201, 400, 401, 403, 404, 500) and error handling.
- **Manual Testing:** Tested edge cases in the UI, such as submitting forms with empty fields, and verifying that the Protected Routes correctly bounce unauthenticated users back to the landing page.

SECTION 17 — DESIGN PATTERNS & SOFTWARE ENGINEERING PRINCIPLES

- **MVC Pattern:** Used heavily in the backend. Controllers handle logic, Models (SQL queries) handle data, and React acts as the View.
- **Separation of Concerns:** Frontend is completely decoupled from the backend. They only communicate via JSON APIs.
- **Singleton Pattern:** The MySQL Database Connection Pool acts as a singleton, shared across all controllers.

SECTION 18 — FUTURE SCOPE & IMPROVEMENTS

1. **Third-Party API Integration:** Integrate Spoonacular or Edamam APIs to automatically fetch macro-nutrients for a recipe based purely on ingredient text.
2. **AI Integration:** Implement OpenAI API to generate dynamic, unique recipes on the fly based on what ingredients the user currently has in their fridge.
3. **Social Features:** Allow users to share their custom recipes and comment on others.

SECTION 19 — FINAL PROJECT SUMMARY

Executive Summary: NutriGuide is a modern, full-stack web application designed to simplify personal nutrition. Built using the PERN-style stack (MySQL instead of Postgres), it features robust JWT authentication, role-based access control, interactive data visualization via Recharts, and a custom rule-based recommendation engine. The project demonstrates strong proficiency in React state management, RESTful API design, relational database architecture, and secure software development practices.

Resume-Ready Bullet Points: - Developed a full-stack health tracking SPA using React.js, Node.js, Express, and MySQL. - Implemented stateless JWT authentication and Role-Based Access Control (RBAC) for admin/user routing. - Engineered a rule-based algorithm in Node.js to filter and recommend recipes based on user diet goals and dynamic allergen parsing. - Integrated `Recharts` to visualize historical caloric data, and utilized `mysql2` connection pooling for optimized database querying.

Generated by AI Assistant. Save this file and use an extension like 'Markdown PDF' in VS Code to export as a formal PDF document.